

RZ-Flow: Readout-Zone and Feedforward-Aware Compilation for Dynamic Zoned Neutral-Atom Quantum Computers

Yifeng Peng*
Stevens Institute of Technology
Hoboken, New Jersey, USA
ypeng21@stevens.edu

Xinyi Li*
Stevens Institute of Technology
Hoboken, New Jersey, USA
xli215@stevens.edu

Yidong Zhou
Rutgers University
New Brunswick, New Jersey, USA
yidong.zhou@rutgers.edu

Siyuan Niu
University of Central Florida
Orlando, Florida, USA
Siyuan.niu@ucf.edu

Hao Wang
Stevens Institute of Technology
Hoboken, New Jersey, USA
hwang9@stevens.edu

Bo Yuan
Rutgers University
New Brunswick, New Jersey, USA
bo.yuan@soe.rutgers.edu

Ying Wang
Stevens Institute of Technology
Hoboken, New Jersey, USA
ywang6@stevens.edu

Abstract

Dynamic zoned neutral-atom quantum computers increasingly support mid-circuit readout, ancilla reuse, and real-time feedforward, turning readout from a terminal I/O step into a recurring, capacity-constrained shared architectural service. Existing zoned neutral-atom compilers, however, remain largely movement-centric and do not explicitly manage readout contention or hide feedback latency, leaving substantial exposed feedforward stall on the execution critical path. We present RZ-Flow, a readout-zone and feedforward-aware compiler for dynamic zoned neutral-atom systems with AOD-constrained transport and finite readout capacity. RZ-Flow addresses this bottleneck through three coordinated mechanisms. It first extracts measurement frontiers to identify work that is truly feedforward-dependent and work that can be released early. It then reserves scarce readout slots and pre-stages soon-to-be-measured qubits near readout. Finally, it overlaps imaging and decoding with legally independent quantum work to reduce exposed feedforward stall. Across 865 dynamic benchmark instances, RZ-Flow improves the makespan by 1.24 $\times$, reduces exposed feedforward stall by 58.1%, and improves expected fidelity proxy by 1.05% over Barrier-Greedy. Even against the per-instance strongest-baseline envelope, it retains a 1.01 $\times$ makespan speedup and a 6.8% stall reduction with modest compile-time overhead. These results establish readout-aware compilation as a distinct and increasingly critical systems layer for scalable dynamic neutral-atom computing.

CCS Concepts

• **Software and its engineering** → **Compilers**; • **Hardware** → **Quantum computation**.

*Both authors contributed equally to this research.



This work is licensed under a Creative Commons Attribution 4.0 International License. ICCAD '26, San Jose, CA, USA

© 2026 Copyright held by the owner/author(s).
ACM ISBN 979-8-4007-2873-0/2026/11
<https://doi.org/10.1145/3831252.3833961>

Keywords

Quantum compilation, zoned neutral-atom architectures, feedforward-aware scheduling, readout-aware compilation

ACM Reference Format:

Yifeng Peng, Xinyi Li, Yidong Zhou, Siyuan Niu, Hao Wang, Bo Yuan, and Ying Wang. 2026. RZ-Flow: Readout-Zone and Feedforward-Aware Compilation for Dynamic Zoned Neutral-Atom Quantum Computers. In *IEEE/ACM International Conference on Computer-Aided Design (ICCAD '26)*, November 08–12, 2026, San Jose, CA, USA. ACM, New York, NY, USA, 9 pages. <https://doi.org/10.1145/3831252.3833961>

1 Introduction

Dynamic quantum circuits, which combine mid-circuit measurement with real-time classical feedforward, are becoming an important execution model for neutral-atom quantum computers. Their canonical motivation includes fault-tolerant workflows such as quantum error correction [30], but the same pattern also appears in teleportation-style protocols and algorithmic primitives such as semiclassical quantum Fourier transform and iterative phase estimation [16]. Neutral-atom quantum computing is therefore entering a qualitatively new systems regime. What were once viewed as device-level advantages—coherent atom transport, reconfigurable connectivity, and spatially separated operating regions—are rapidly becoming compiler-visible architectural resources. Recent experiments and architecture proposals have demonstrated coherent transport of entangled atom arrays, logical processing on zoned reconfigurable atom arrays, and fault-tolerant neutral-atom designs that explicitly expose storage, entangling, and readout regions as distinct resources [2–4]. At the same time, mid-circuit measurement, reset, ancilla reuse, and real-time feedforward are evolving from isolated demonstrations into durable runtime primitives on neutral-atom hardware [5, 12, 14, 15]. For the compiler, the implication is fundamental: measurement is no longer a terminal I/O event appended to the end of a circuit. In dynamic zoned processors, it becomes a recurring, capacity-constrained, latency-bearing service that can repeatedly re-enter the execution critical path. Figure 1 previews this effect: even a movement-efficient schedule can still

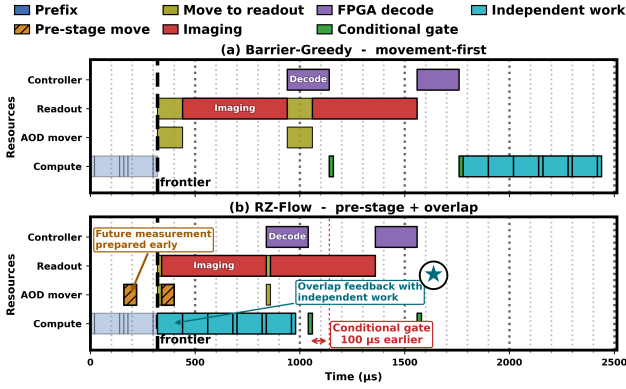


Figure 1: Motivating example: movement-first scheduling can still leave substantial feedforward latency exposed. (a) A movement-first baseline delays readout preparation until measurement becomes urgent, so move-to-readout, imaging, and decode remain largely serialized on the critical path. (b) RZ-Flow pre-stages future measurements and overlaps the feedback interval with independent work, reducing the exposed measurement-to-action delay.

expose substantial readout and feedforward latency on the critical path during repeated dynamic execution.

Rapid advances in quantum computing continue to broaden its algorithmic capabilities and application landscape [18, 19]. At the systems layer, a rich neutral-atom CAD stack has emerged. Architecture and cross-layer studies have clarified how long-distance interactions, atom mobility, atom-loss tolerance, and entangling-zone geometry reshape the design space [1, 10, 11]. Compiler work has advanced from early neutral-atom frameworks such as GEYSER [17], to reconfigurable-array compilers such as ATOMIQUE [27] and Q-PILOT [28], to mapping/topology co-optimization such as hybrid circuit mapping [22]. More recent zoned and movement-aware compilers, including ZAC [13], ENOLA [26], POWERMOVE [21], movement-optimized zoned program generation [7], routing-aware zoned placement [24], and recent directional-transport and buffer-relay approaches [6, 8], have substantially improved qubit reuse, placement, routing, and transport scheduling. These advances are important, but they remain largely movement-centric: they optimize how qubits reach entangling opportunities, not how measurement demand is admitted into a constrained readout structure, how controller-side latency is hidden, or how post-measurement independent work is released once dynamic control enters the loop.

That gap becomes first-order for circuits with repeated mid-circuit measurement and feedforward. Without dynamic control, movement-centric zoned compilation is often sufficient. Once dynamic control becomes frequent, however, the compiler must decide not only how qubits move, but also when qubits should leave computation for readout, how scarce readout service should be allocated, and which operations can proceed before feedback resolves. Consequently, a schedule that is near-optimal for movement can still be poor end-to-end: it may deliver measurement traffic too late,

serialize readout-side latency, or leave the compute fabric idle while waiting for outcomes.

We introduce RZ-FLOW, a readout-zone and feedforward-aware compiler for dynamic zoned neutral-atom quantum computers. Rather than treating measurement as an afterthought on top of a movement-centric schedule, RZ-FLOW elevates it to a first-class architectural event and jointly optimizes measurement-frontier extraction, readout-slot reservation, anticipatory evacuation, and latency-hiding scheduling. Across 865 benchmark instances spanning six dynamic-kernel families, including IQPE, scQFT, teleportation, and ancilla reuse, RZ-FLOW achieves a geometric-mean makespan speedup of 1.24 $\times$, reduces exposed feedforward stall (EFS) by 58.1%; even under the per-instance strongest-baseline envelope, it still retains a 1.01 $\times$ speedup and a 6.8% EFS reduction with only modest compile-time overhead. Our contributions are summarized below:

- We identify readout flow as a compiler-visible bottleneck in dynamic zoned neutral-atom execution, and argue that exposed feedforward stall, rather than movement overhead alone, should be treated as the primary optimization target.
- We introduce RZ-FLOW, a readout-zone and feedforward aware compilation framework with a unified scheduling loop that jointly reasons about measurement frontiers, scarce readout admission, anticipatory evacuation, and latency-hiding release of independent work.
- We provide an end-to-end evaluation focused on readout-aware dynamic zoned compilation, showing robust gains across six workload families and multiple movement-centric baselines, while remaining essentially non-intrusive on static workloads.
- We expose a compiler/architecture co-design insight: once dynamic control becomes frequent, improving the readout and feedback path can yield larger system-level benefit than adding more entangling-side resources, and the compiler materially changes the marginal value of such hardware upgrades.

2 Background and Motivation

Readout Has Become a First-Class Architectural Resource. Dynamic zoned neutral-atom processors elevate readout from terminal I/O to a scheduled architectural resource. Readout is not uniformly available. Measurements must flow through a few dedicated readout zones with limited throughput and non-negligible controller latency, making readout admission a first-order scheduling problem. Storage, entangling, and readout regions now expose different capacities and movement constraints, and recent logical and fault-tolerant systems support selective mid-circuit measurement while preserving coherence elsewhere in the array [2, 3]. A measurement, therefore, includes not only physical readout, but also admission, imaging, controller-side transfer, logical-state decoding, and conditional release [2].

This is no longer rare. Mid-circuit measurement, reset, and conditional execution have been demonstrated across neutral-atom platforms [5, 14], and repeated runtime loops with ancilla reuse, qubit replacement, and sustained feedback are emerging [3, 12, 15].

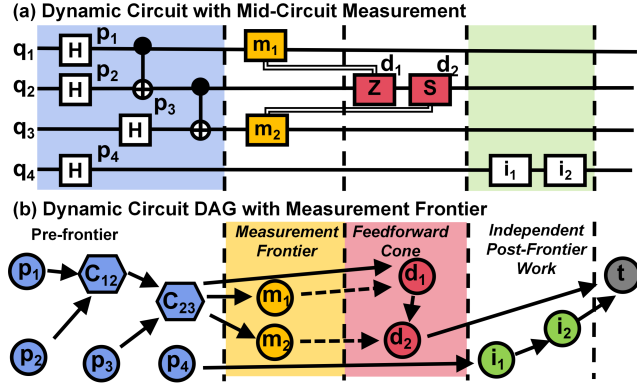


Figure 2: Measurement frontiers expose overlap. (a) A dynamic circuit with two pre-frontier two-qubit operations (C_{12} and C_{23}), a measurement frontier $\{m_1, m_2\}$, feedforward-dependent operations (d_1, d_2), and independent post-frontier work (i_1, i_2). (b) The corresponding precedence DAG.

For dynamic circuits, readout demand becomes bursty and capacity-limited, so the compiler must decide when qubits leave coherent computation, how readout slots are allocated during execution, and how much feedback latency can be hidden behind independent work. Readout is thus a first-class architectural resource.

Why Movement-Centric Compilation Becomes Insufficient. Related work has made mid-circuit-measurement errors visible to layout, routing, and scheduling [29], while prior zoned neutral-atom compilers primarily optimize movement through placement, routing, qubit reuse, zone switching, and transport scheduling [7, 13, 21, 23, 24]. These techniques remain necessary, but they are no longer sufficient once mid-circuit measurement and feedforward repeatedly enter the critical path. Dynamic execution couples transport, readout service, and classical dependence resolution. A schedule can therefore be movement-efficient yet still expose long stalls if measurements are sent to readout only when urgent, if readout slots are claimed greedily under contention, or if independent post-measurement work is conservatively blocked.

Figure 1 illustrates the mismatch. BARRIER-GREEDY is locally movement-efficient, but delays readout preparation and serializes move-to-readout, imaging, and decode on the critical path. RZ-Flow improves latency not simply by moving less, but by surfacing measurement demand earlier and overlapping feedback with legally independent work. This motivates exposed feedforward stall, defined as the uncovered portion of the readout interval that remains on the critical path.

Figure 2 makes this key observation concrete: once a measurement frontier is identified, the post-frontier DAG decomposes into a true feedforward cone and a releasable set of independent work. Measurement frontiers are therefore the key abstraction in RZ-Flow: they separate truly feedforward-dependent work from work that can be legally released early. This separation is what creates an overlap opportunity and motivates EFS as the right optimization target for dynamic zoned execution.

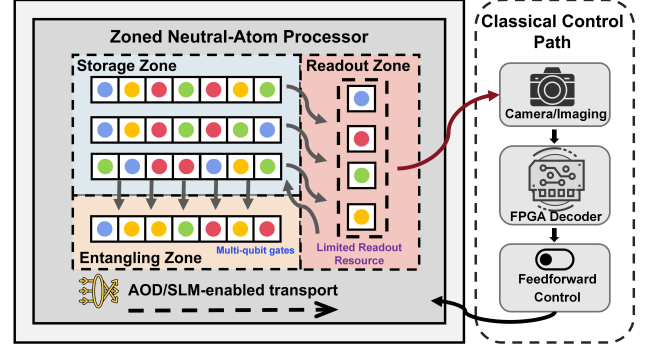


Figure 3: Target zoned neutral-atom architecture. Readout is modeled as a finite-capacity shared service with admission, imaging, transfer, decode, and release latency.

3 Proposed: RZ-Flow

RZ-Flow optimizes EFS rather than movement alone. On top of movement-aware placement and routing, RZ-Flow adds three mechanisms: measurement-frontier extraction, reserved readout admission with anticipatory evacuation, and latency-hiding scheduling.

Overview. RZ-Flow does not eliminate readout latency; instead, it reduces the unavoidable portion and hides the remainder behind useful independent work. The key departure from prior movement-centric zoned compilers is therefore not only how qubits are brought to entangling zones efficiently, but how measurement demand is surfaced, admitted, and overlapped so that dynamic control exposes less stall. Figure 3 summarizes the target zoned neutral-atom architecture and the readout/control path that RZ-Flow treats as a finite-capacity shared service.

Let the dynamic circuit be a precedence DAG $G = (V, E_q \cup E_c)$, where E_q denotes quantum dependences and E_c denotes classical dependences induced by measurements. We extract a topologically ordered sequence of measurement frontiers $\{M_j\}_j$, where each $M_j \subseteq V$ is a set of measurements whose outcomes may release subsequent work. For frontier j , let $P_j \subseteq V$ denote the operations after the prefix ending at M_j . We partition P_j into a feedforward cone and a releasable set:

$$\text{dep}_j = \{u \in P_j \mid \exists m \in M_j \text{ such that } m \rightsquigarrow_c u\}, \quad (1)$$

where $m \rightsquigarrow_c u$ denotes reachability from m to u along a path containing at least one edge in E_c , and $\text{ind}_j = P_j \setminus \text{dep}_j$. Thus, dep_j contains the post-frontier operations that truly require measurement outcomes, whereas ind_j contains the operations that may be legally released early.

When $\text{dep}_j = \emptyset$, we set $G_j = \text{EFS}_j = 0$. Otherwise, we summarize frontier j through its earliest newly releasable dependent boundary:

$$R_j = \{u \in \text{dep}_j \mid \text{pred}(u) \cap \text{dep}_j = \emptyset\}. \quad (2)$$

For each $u \in R_j$, let $M_j(u) = \{m \in M_j \mid m \rightsquigarrow_c u\}$ denote the subset of frontier measurements required by u . Let $r(u)$ be the earliest feasible issue time of u , assuming that all outcomes in $M_j(u)$ were already available and all non-classical predecessors had completed.

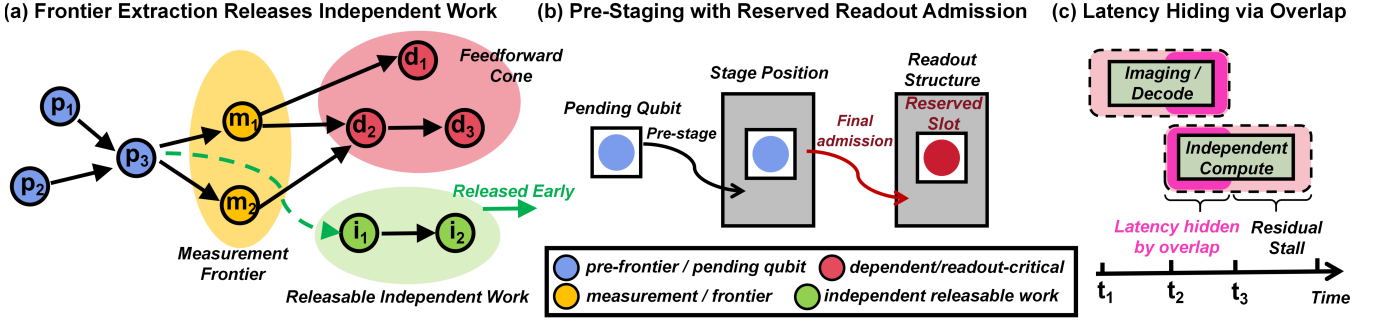


Figure 4: Core mechanisms of RZ-FLOW. (a) Frontier extraction releases independent post-measurement work. (b) Reserved admission and anticipatory evacuation pre-stage soon-to-be-measured qubits near readout. (c) Latency-hiding scheduling overlaps feedback with released work to reduce EFS.

Let $c(m)$ be the time at which measurement m becomes usable, and define $c(u) = \max_{m \in M_j(u)} c(m)$.

We summarize the frontier j through the earliest releasable boundary operation $u_j^* \in \arg \min_{u \in R_j} \max(r(u), c(u))$, with paired frontier times $r_j = r(u_j^*)$, $c_j = c(u_j^*)$. The raw feedback gap and residual exposed stall are

$$G_j = [c_j - r_j]_+, \text{EFS}_j = [G_j - B_j]_+, [x]_+ = \max(x, 0), \quad (3)$$

where B_j is the effective wall-clock overlap achieved by operations from ind_j inside $[r_j, c_j]$, measured in time rather than aggregate work volume. The aggregate exposed stall is $\text{EFS} = \sum_j \text{EFS}_j$, used as a frontier-level scheduling surrogate rather than an exact additive decomposition of makespan. RZ-FLOW minimizes EFS jointly with makespan, $\min C_{\max} + \lambda \sum_j \text{EFS}_j$, subject to dependency legality, zone-capacity constraints, AOD movement constraints, and finite readout capacity. Here $\lambda \geq 0$ is a trade-off weight.

3.1 Readout as a Shared Service

Measurements must enter a finite-capacity readout structure. For each measurement $m \in M_j$, let a_m be the arrival time of the measured qubit(s), w_m the reserved opening time of the assigned readout slot, and L_R be the service latency, including imaging, transfer, decode, and release. Then $c(m) = \max(a_m, w_m) + L_R$. Substituting this into the selected-boundary definition:

$$c(u) = \max_{m \in M_j(u)} (\max(a_m, w_m) + L_R),$$

and therefore

$$G_j = \left[\max_{m \in M_j(u_j^*)} (\max(a_m, w_m) + L_R) - r(u_j^*) \right]_+. \quad (4)$$

Eq. (4) exposes the compiler levers that act directly on the raw gap: anticipatory evacuation reduces the relevant arrival times a_m , and reservation reduces waiting at the readout boundary through w_m . Latency-hiding scheduling does not change G_j ; instead, it increases the realized overlap budget B_j and therefore reduces the residual stall $\text{EFS}_j = [G_j - B_j]_+$.

Algorithm 1: RZ-FLOW

```

1 def RZFLOW( $G, \mathcal{A}$ ):
2    $F \leftarrow \text{ExtractFrontiers}(G)$ 
3   //  $F := (\{M_j\}_j, \{\text{dep}_j, \text{ind}_j\}_j)$ 
4    $W \leftarrow \text{ReserveReadout}(F, \mathcal{A})$ 
5   initialize time  $t$  and schedule  $S$ 
6   while unscheduled operations remain do
7      $\text{StagePending}(W, \mathcal{A}, t)$ 
8      $R(t) \leftarrow \text{ReleasedReady}(G, F, S, t)$ 
9     if  $R(t) \neq \emptyset$  then
10       $o^* \leftarrow \text{PickBest}(R(t), t)$  using Eq. (5)
11       $\text{Issue}(o^*, \mathcal{A}, S, t)$ 
12    else
13       $t \leftarrow \text{NextEvent}(\mathcal{A}, t)$ 
14  return  $S$ 

```

3.2 Scheduling Rule

At time t , let $R(t)$ be the set of ready operations that satisfy dependency and resource legality. RZ-FLOW selects

$$o^*(t) = \arg \max_{o \in R(t)} \left(\alpha \text{Crit}(o) + \beta \text{Hide}(o, t) - \gamma \text{Miss}(o, t) + \delta \log \tilde{\pi}(o) \right), \quad (5)$$

where $\alpha, \beta, \gamma, \delta \geq 0$ are scheduler weights, $\text{Crit}(o)$ is downstream criticality, $\text{Hide}(o, t)$ estimates stall-hiding benefit, and $\text{Miss}(o, t)$ penalizes interference with imminent reserved admissions. Here $\pi(o) \in [0, 1]$ denotes the execution probability of operation o under the current branch model, and $\tilde{\pi}(o) = \max\{\pi(o), \varepsilon\}$ with a small constant $\varepsilon > 0$. When branch information is unavailable, or when o is not branch specific, we set $\pi(o) = 1$.

3.3 End-to-End Scheduling Loop

Algorithm 1 summarizes the full loop. RZ-FLOW first extracts measurement frontiers and reserves readout slots. It then repeatedly pre-stages pending measurements, constructs the released-ready set, schedules the highest-score legal operation, and otherwise

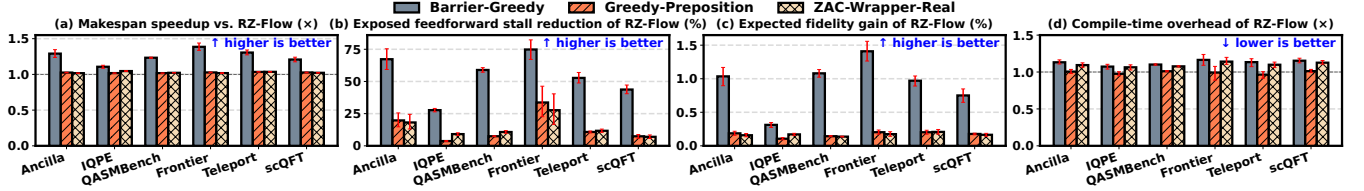


Figure 5: End-to-end benefit across workload families. RZ-Flow reduces end-to-end makespan, lowers exposed feedforward stall (EFS), and increases expected fidelity proxy across six dynamic-kernel families. (a) Baseline-to-RZ-Flow makespan speedup. (b) EFS reduction. (c) Fidelity gain. (d) Compile overhead. Error bars show 95% bootstrap confidence intervals.

Table 1: Aggregate summary. RZ-Flow improves the makespan, reduces EFS, and increases expected fidelity proxy across baselines. Makespan and compile overhead are geometric means; stall reduction is reported on valid non-zero-stall instances.

Comparison	Makespan speedup $\uparrow$ (x)	Stall reduction [†] $\uparrow$ (%)	Fidelity gain $\uparrow$ (%)	Compile OH $\downarrow$ (x)	MKSP W/T/L	FID W/T/L	Stall valid n
Barrier-Greedy	1.237 [1.21, 1.26]	58.11 [54.3, 61.8]	1.054 [0.96, 1.15]	1.105 [1.08, 1.13]	865/0/0	865/0/0	865
Greedy-Preposition	1.019 [1.01, 1.03]	8.72 [7.1, 10.4]	0.148 [0.13, 0.17]	1.008 [0.99, 1.03]	709/153/3	865/0/0	713
ZAC-Wrapper-Real	1.023 [1.01, 1.03]	11.37 [9.6, 13.2]	0.143 [0.12, 0.16]	1.081 [1.06, 1.10]	689/146/30	865/0/0	722
Strongest-baseline envelope	1.014 [1.01, 1.02]	6.81 [5.4, 8.1]	0.123 [0.11, 0.14]	1.130 [1.10, 1.16]	677/155/33	865/0/0	713

[†] Valid-only: instances whose comparison baseline exposes non-zero feedforward stall.

W/T/L denotes win/tie/loss counts of RZ-Flow relative to the comparison baseline.

advances time to the next event. In this way, movement, readout admission, and dependence release are coupled in a single scheduler. Here $\mathcal{A}$ denotes the architectural state and resource model.

Mechanism Summary. Frontier extraction enlarges the releasable set ind_j and sharpens the boundary R_j . Reserved admission and anticipatory evacuation reduce the raw gap in Eq. (4). Latency-hiding scheduling increases B_j and therefore reduces EFS_j . Figure 4 summarizes how RZ-Flow operationalizes this objective through three mechanisms.

4 Evaluation

We first ask whether readout-aware compilation improves end-to-end execution over strong movement-centric baselines. We then isolate the mechanisms that produce the gain, identify the workload and hardware regimes in which the gain is largest, test the robustness of branch-aware expected-case compilation, and finally study scalability, backward compatibility, and compiler-aware architecture co-design.

4.1 Experimental Setup

4.1.1 Hardware Settings. All methods use the zoned neutral-atom model of Section 2, with storage, entangling, and finite-capacity readout regions and AOD-constrained transport [2, 3]. Operation latencies are 20/110 μs for one-/two-qubit gates, 60 μs for rearrangement, 120 μs for direct readout transport, 80 μs for pre-staging, 20 μs for admission, 500 μs for imaging, and 180 μs for decoding [20, 25].

4.1.2 Baselines. BARRIER-GREEDY inserts a full barrier at every measurement frontier. GREEDY-PREPOSITION pre-stages measured qubits using local greedy priorities but lacks frontier-aware criticality. ZAC-WRAPPER-REAL partitions the dynamic DAG at measurement frontiers and applies a ZAC-inspired critical-path cost model

to each static segment [13]. All methods share identical architecture and timing assumptions.

4.1.3 Benchmarks. The primary suite contains 865 paired dynamic instances in six families. 740 instances from 148 QASM circuits spanning 80 source subfamilies [9]; we render these static circuit structures dynamic by injecting mid-circuit measurement and feed-forward under a fixed execution model. The other 125 instances comprise 25 each from AncillaReuse, IQPE, SyntheticFrontier, Teleportation, and scQFT. The suite covers 3–128 qubits and 3–27 mid-circuit measurements. Unless explicitly varied, results report 95% bootstrap confidence intervals.

4.1.4 Metrics. Our primary performance metric is **makespan** $\downarrow$, reported as baseline-to-RZ-Flow **speedup** $\uparrow$ for comparative summaries. Our primary quality metric is **expected fidelity proxy**¹ $\uparrow$, evaluated under the same timing and movement model for all methods. To expose the mechanism, we report **EFS** $\downarrow$ and **EFS reduction** $\uparrow$, together with **compile time** $\downarrow$ and **peak memory** $\downarrow$.

4.2 Experimental Results

End-to-end results across workload families. Illustrated in Fig. 5 and Table 1, RZ-Flow delivers cross-family, end-to-end gains. Against BARRIER-GREEDY, RZ-Flow achieves a geometric-mean makespan speedup of 1.237 $\times$, reduces EFS by 58.11%, and improves expected fidelity proxy by 1.054%. The gap narrows but remains positive against stronger baselines, RZ-Flow still delivers 1.019 $\times$ –1.023 $\times$ makespan speedup, 8.72%–11.37% EFS reduction, and 0.143%–0.148% fidelity gain relative to GREEDY-PREPOSITION and ZAC-WRAPPER-REAL. The family-wise bars in Fig. 5 show that this advantage recurs

¹The expected fidelity proxy is computed from the scheduled trace using the same calibrated trace-level estimator for all methods; the estimator maps gate, transport, readout, and idle exposure to a single success probability.

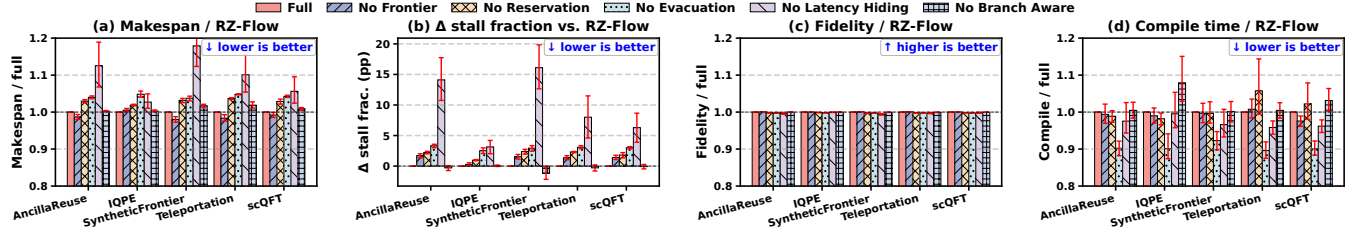


Figure 6: Ablation identifies latency hiding as the dominant mechanism. (a) Normalized makespan. (b) Increase in exposed-stall fraction relative to full RZ-Flow. (c) Normalized expected fidelity proxy. (d) Normalized compile time.

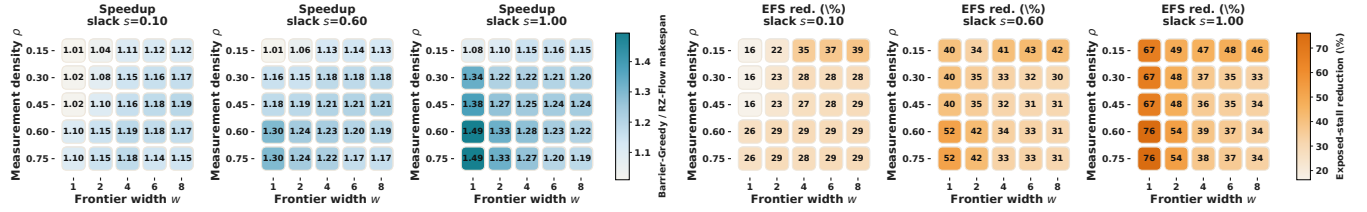


Figure 7: Regime sweep over measurement density and frontier width. RZ-Flow becomes most valuable when readout pressure and overlap opportunity coexist. Left: makespan speedup over BARRIER-GREEDY. Right: EFS reduction. Rows and columns sweep measurement density ρ , frontier width w , and independent slack s .

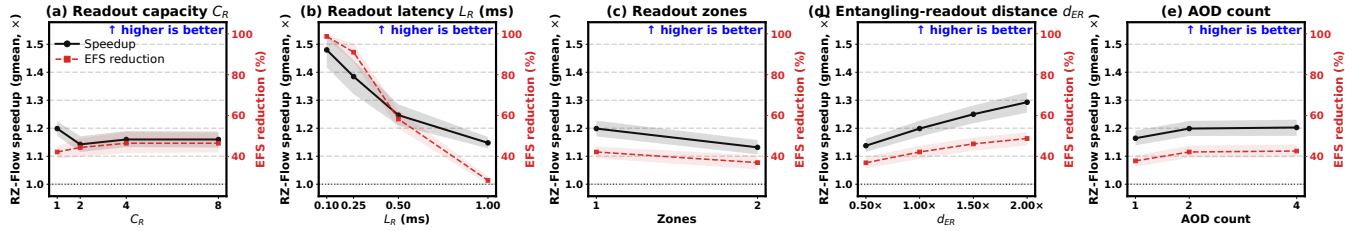


Figure 8: Aggregate hardware sensitivity. RZ-Flow remains beneficial across the hardware sweep, with the largest response in the readout-latency setting. Black: geometric-mean makespan speedup over BARRIER-GREEDY. Red: EFS reduction. Shaded bands show 95% bootstrap confidence intervals.

across ancilla-driven, iterative, teleportation-style, and frontier-heavy kernels, not limited to one benchmark type.

Table 1 shows that the result is also conservative under stronger comparison rules. Even under the strongest-baseline envelope, RZ-Flow retains a 1.014 $\times$ speedup, 6.81% EFS reduction, and 0.123% fidelity gain, with 677/155/33 win/tie/loss on makespan and 865/0/0 on fidelity. Thus, the gain is broad-based and not driven by a few outliers. At the same time, Fig. 5(d) shows that compile overhead ranges from near parity to the low teens, so the improvement is truly end-to-end compared to a shift of cost into compilation.

Which mechanisms create the gain? Figure 6 shows that RZ-Flow’s gain is systematic. All major components matter, but latency hiding is the dominant one: removing it produces the largest exposed-stall increase and the largest makespan regression across families. This directly supports our central claim that, for dynamic zoned execution, the main avoidable cost is unhidden measurement/feedback latency. Reservation and anticipatory evacuation are the next most important, as they smooth readout admission

and enlarge overlap opportunities. Frontier extraction and branch-aware prioritization provide smaller, workload-dependent refinements. Fidelity remains nearly unchanged and compile overhead modest (Figs. 6(c)–6(d)), confirming that the gain comes from better stall suppression rather than a different quality-cost tradeoff.

When does readout-aware compilation matter? Figure 7 reveals a clear regime boundary. When measurement density ρ and frontier width w are both small, RZ-Flow yields only limited improvement, indicating that readout is not yet the dominant bottleneck. As either factor increases—especially together—the benefit rises rapidly, and larger independent slack s amplifies the gain by creating more overlap opportunity along the feedback path. This directly matches the intended mechanism: readout-aware compilation matters most when readout contention is significant and sufficient post-frontier work exists to hide feedback latency. In the strongest regime, RZ-Flow reaches about 1.49 $\times$ makespan speedup and roughly 76% EFS reduction, confirming that readout flow becomes a first-order optimization target once pressure and overlap opportunity coexist.

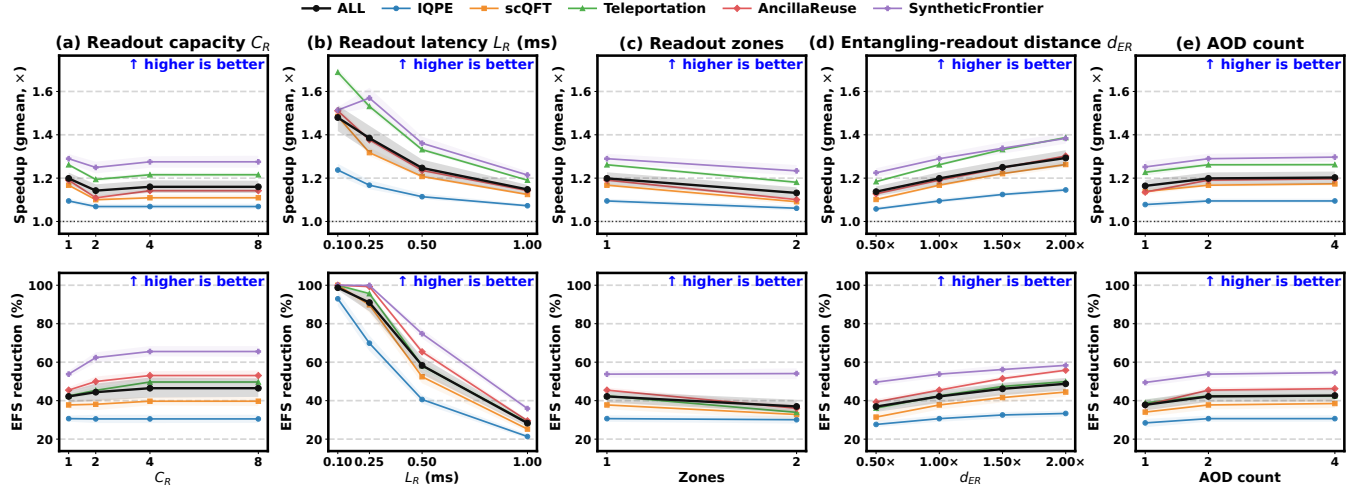


Figure 9: Hardware sensitivity across workload families. The hardware-dependence trend of RZ-Flow persists across families. Top row: makespan speedup over BARRIER-GREEDY. Bottom row: EFS reduction. Columns sweep readout capacity, readout latency, readout zones, entangling-readout distance, and AOD count.

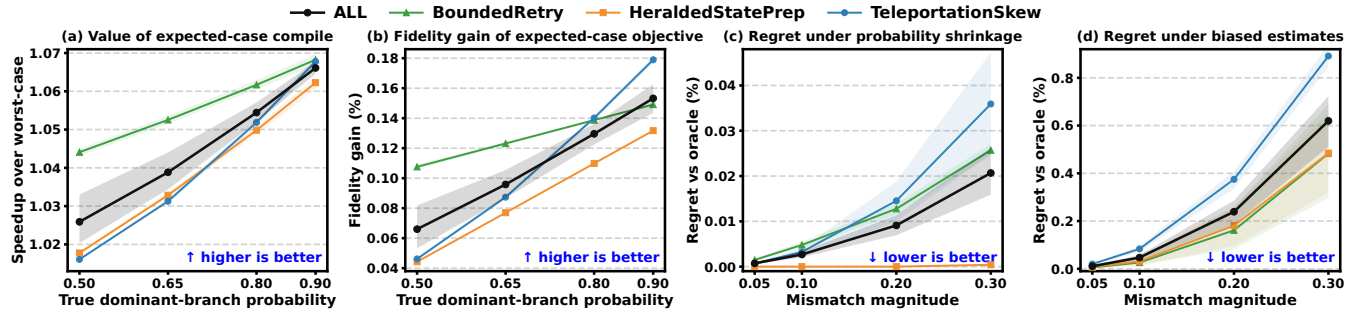


Figure 10: Value and robustness of branch-aware expected-case compilation. Expected-case compilation becomes more beneficial as branch skew increases and remains robust to moderate probability mismatch. (a) Average-case speedup over the worst-case objective. (b) Fidelity gain. (c,d) Regret relative to an oracle under two mismatch models.

How does the gain vary with hardware? Figures 8 and 9 show that RZ-Flow remains effective across all tested hardware settings, but its benefit scales with the severity of the readout bottleneck. The strongest sensitivity appears in readout latency, identifying controller-side delay as the most effective hardware knob. Adding a readout zone reduces the gain by easing contention, whereas increasing entangling-readout distance enlarges it by making measurement traffic more expensive. Readout capacity and AOD count show weaker but still consistent effects. These trends persist across workload families, with speedup and EFS closely tracking each other. The implication is clear: RZ-Flow is robust across hardware settings, yet most valuable precisely when the readout/feedforward path becomes the dominant system bottleneck.

Is branch-aware expected-case compilation useful and robust? Figure 10 shows that the optional branch-aware objective is both useful and stable on branch-skewed workloads. As the dominant-branch probability increases, expected-case compilation delivers steadily

larger average-case makespan and fidelity gains over the worst-case objective, confirming that exploiting branch skew improves the realized schedule. The robustness results are similarly encouraging. Regret remains very small under probability shrinkage, and although biased estimates cause larger degradation on strongly asymmetric workloads, the aggregate loss stays controlled. Overall, branch-aware compilation adds clear value when branch skew exists and degrades gracefully when branch probabilities are imperfect or only approximately known.

Is RZ-Flow practical at scale and safe on static circuits? Figure 11 and Table 2 show that RZ-Flow is both scalable and non-intrusive. As dynamic depth and problem size grow, compile time and peak memory increase smoothly, while compile overhead over BARRIER-GREEDY remains close to unity. At the same time, solution quality remains positive and generally improves with scale, indicating that the method remains practical rather than becoming a fragile high-overhead optimizer. Backward compatibility is similarly strong.

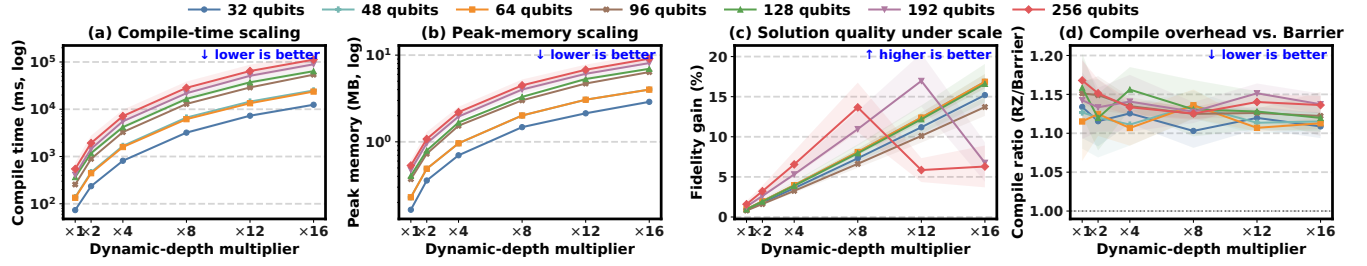


Figure 11: Scalability and compile overhead. RZ-Flow scales with dynamic depth and system size while maintaining modest compile overhead. (a) Compile time. (b) Peak memory. (c) Solution quality. (d) Compile ratio relative to BARRIER-GREEDY.

Table 2: Backward compatibility on static circuits. On static GHZ, Ising/QAOA-style, and parallel-CZ workloads, RZ-Flow is effectively non-intrusive, with negligible change in makespan, fidelity, compile time, and peak memory relative to BARRIER-GREEDY.

Workload	Δ Makespan (%)	Δ Fidelity (%)	Δ Compile (%)	Δ Peak Mem. (%)	Resource cap.*
GHZ	0.00%	0.00%	-0.03%	-0.01%	70
Ising / QAOA	0.00%	0.00%	+0.14%	0.00%	70
Parallel CZ	0.00%	0.00%	-0.04%	0.00%	70

*: Per-group service capacity (Drive/Coupler/Readout), i.e., the maximum number of operations that a resource group can support concurrently.

On static GHZ, Ising/QAOA-style, and parallel-CZ workloads, RZ-Flow introduces essentially no makespan or fidelity regression and only negligible compile-time or memory change. Thus, RZ-Flow scales well when dynamic complexity increases and remains nearly invisible when dynamic control is absent.

How does the compiler change architecture decisions? Figure 12 shows that upgrade value depends on both workload mix and compiler quality. For dynamic-heavy workloads, readout-side upgrades are most valuable, especially lower readout latency; for static-heavy workloads, the benefit shifts toward a wider entangling zone. Compared with BARRIER-GREEDY, RZ-Flow further increases the value of faster readout while reducing the marginal benefit of extra zone capacity, since part of the contention and overlap bottleneck is already removed in software.

Overall, the experiments consistently show that RZ-Flow improves end-to-end execution by suppressing exposed feedforward stall, remains robust across regimes, and stays practical without harming static workloads. This makes readout-aware compilation a distinct systems layer for dynamic zoned neutral-atom execution.

5 Discussion

Our results show that readout flow is a first-order compilation concern in dynamic zoned neutral-atom systems. Once mid-circuit measurement and feedforward become frequent, performance depends not only on movement, but also on how readout demand is admitted and how much feedback latency can be hidden. Our scope is intentionally limited to zoned transport, finite readout service, and controller-side latency, rather than pulse-level control or full logical error analysis. Even so, the main conclusion is clear:

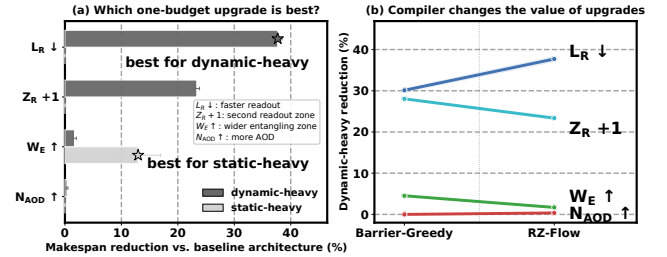


Figure 12: Compiler-aware architecture co-design. (a) Dynamic-heavy workloads favor readout-side upgrades, whereas static-heavy workloads favor a wider entangling zone. (b) Compared with BARRIER-GREEDY, RZ-Flow increases the value of faster readout while reducing the return of extra zone capacity.

exposed feedforward stall is a compiler-visible bottleneck and a key optimization target for dynamic neutral-atom execution.

6 Conclusion

We presented RZ-Flow, a readout-zone and feedforward-aware compiler for dynamic zoned neutral-atom quantum computers. By treating readout as a first-class shared architectural service, RZ-Flow shifts the optimization target from movement alone to exposed feedforward stall (EFS), combining frontier extraction, reserved admission, anticipatory evacuation, and latency-hiding scheduling. Across diverse dynamic workloads, RZ-Flow consistently improves makespan, reduces EFS, and increases expected fidelity proxy over strong movement-centric baselines; even under the strongest-baseline envelope, it still retains measurable gains in all three metrics. These results show that readout-aware compilation is not a narrow heuristic, but a distinct systems layer whose importance will grow as neutral-atom platforms expose richer mid-circuit measurement, reuse, and feedforward capabilities.

Acknowledgments

This work was partially supported by the U.S. National Science Foundation (NSF) under Grant No. 2555385. Any opinions, findings, and conclusions or recommendations expressed in this material are those of the authors and do not necessarily reflect the views of the U.S. National Science Foundation.

References

- [1] Jonathan M Baker, Andrew Litteken, Casey Duckering, Henry Hoffmann, Hannes Bernien, and Frederic T Chong. 2021. Exploiting long-distance interactions and tolerating atom loss in neutral atom quantum architectures. In *2021 ACM/IEEE 48th Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 818–831.
- [2] Dolev Bluvstein, Simon J Evered, Alexandra A Geim, Sophie H Li, Hengyun Zhou, Tom Manovitz, Sepehr Ebadi, Madelyn Cain, Marcin Kalinowski, Dominik Hangleiter, et al. 2024. Logical quantum processor based on reconfigurable atom arrays. *Nature* 626, 7997 (2024), 58–65.
- [3] Dolev Bluvstein, Alexandra A Geim, Sophie H Li, Simon J Evered, J Pablo Bonilla Ataides, Gefen Baranes, Andi Gu, Tom Manovitz, Muqing Xu, Marcin Kalinowski, et al. 2026. A fault-tolerant neutral-atom architecture for universal quantum computation. *Nature* 649, 8095 (2026), 39–46.
- [4] Dolev Bluvstein, Harry Levine, Giulia Semeghini, Tout T Wang, Sepehr Ebadi, Marcin Kalinowski, Alexander Keesling, Nishad Maskara, Hannes Pichler, Markus Greiner, et al. 2022. A quantum processor based on coherent transport of entangled atom arrays. *Nature* 604, 7906 (2022), 451–456.
- [5] Trent M Graham, Linipun Phuttitarn, Ravikumar Chinnarasu, Yunheung Song, Cody Poole, Kais Jooya, Jacob Scott, Abraham Scott, Patrick Eichler, and Mark Saffman. 2023. Midcircuit measurements on a single-species neutral alkali atom quantum processor. *Physical Review X* 13, 4 (2023), 041051.
- [6] Chen Huang, Jingbo Wang, Zhemin Zhang, Ming Zhong, Zhuo Fu, Zhiding Liang, Yuan Sun, and Dong E. Liu. 2026. Lazy-Move Compilation for Neutral-Atom Quantum Computers via a Buffer-Relay Fabric. *arXiv preprint arXiv:2606.31833* (2026).
- [7] Enhyeok Jang, Youngmin Kim, Hyungseok Kim, Seungwoo Choi, Yipeng Huang, and Won Woo Ro. 2025. Qubit Movement-Optimized Program Generation on Zoned Neutral Atom Processors. In *Proceedings of the 23rd ACM/IEEE International Symposium on Code Generation and Optimization*. 459–475.
- [8] Lingyi Kong, Chen Huang, Zhemin Zhang, Yidong Zhou, Xiangyu Ren, Shaochen Li, and Zhiding Liang. 2026. Compiler Framework for Directional Transport in Zoned Neutral Atom Systems with AOD Assistance: A Hybrid Remote CZ Approach. In *Proceedings of the 63rd ACM/IEEE Design Automation Conference*. doi:10.1145/3770743.3804196
- [9] Ang Li, Samuel Stein, Sriram Krishnamoorthy, and James Ang. 2023. Qasmbench: A low-level quantum benchmark suite for nisq evaluation and simulation. *ACM Transactions on Quantum Computing* 4, 2 (2023), 1–26.
- [10] Xinyi Li, Yifeng Peng, and Ying Wang. 2026. Beyond Legal Spacing: A Residual-Aware Characterization of Entangling-Zone Spacing in Neutral-Atom Compilation. *arXiv preprint arXiv:2608.17331* (2026).
- [11] Xinyi Li, Yifeng Peng, and Ying Wang. 2026. Where Atom Loss Lands Matters: Decoder-Aware Risk Deposition in Neutral-Atom QEC. *arXiv preprint arXiv:2608.17913* (2026).
- [12] Rui Lin, You Li, Le-Tian Zheng, Tai-Ran Hu, Si-Yuan Chen, Hong-Ming Wu, Yu-Chen Zhang, Hao-Wen Cheng, Yu-Hao Deng, Zhan Wu, et al. 2026. Sustaining high-fidelity quantum logic in neutral-atom circuits via mid-circuit operations. *arXiv preprint arXiv:2603.01612* (2026).
- [13] Wan-Hsuan Lin, Daniel Bochen Tan, and Jason Cong. 2025. Reuse-aware compilation for zoned quantum architectures based on neutral atoms. In *2025 IEEE International Symposium on High Performance Computer Architecture (HPCA)*. IEEE, 127–142.
- [14] Joanna W Lis, Aruku Senoo, William F McGrew, Felix Rönchen, Alec Jenkins, and Adam M Kaufman. 2023. Midcircuit operations using the omg architecture in neutral atom arrays. *Physical Review X* 13, 4 (2023), 041035.
- [15] JA Muniz, D Crow, H Kim, JM Kindem, WB Cairncross, A Ryou, TC Bohdanowicz, C-A Chen, Y Ji, AMW Jones, et al. 2025. Repeated ancilla reuse for logical computation on a neutral atom quantum computer. *Physical Review X* 15, 4 (2025), 041040.
- [16] Siyuan Niu, Efehan Kokcu, Anupam Mitra, Aaron Szasz, Akel Hashim, Justin Kalloor, Wibe Albert de Jong, Costin Iancu, and Ed Younis. 2024. Ac/dc: Automated compilation for dynamic circuits. *arXiv preprint arXiv:2412.07969* (2024).
- [17] Tirthak Patel, Daniel Silver, and Devesh Tiwari. 2022. Geyser: a compilation framework for quantum computing with neutral atoms. In *Proceedings of the 49th Annual International Symposium on Computer Architecture*. 383–395.
- [18] Yifeng Peng, Xinyi Li, Samuel Yen-Chi Chen, Kaining Zhang, Zhiding Liang, Ying Wang, and Yuxuan Du. 2026. TITAN: A trajectory-informed technique for adaptive parameter freezing in large-scale vqe. *Advances in Neural Information Processing Systems* 38 (2026), 47294–47315.
- [19] Yifeng Peng, Xinyi Li, Zhiding Liang, and Ying Wang. 2025. Qsco: A quantum scoring module for open-set supervised anomaly detection. In *Proceedings of the AAAI Conference on Artificial Intelligence*, Vol. 39. 19884–19894.
- [20] Linipun Phuttitarn, B_M Becker, Ravikumar Chinnarasu, T_M Graham, and Mark Saffman. 2024. Enhanced measurement of neutral-atom qubits with machine learning. *Physical Review Applied* 22, 2 (2024), 024011.
- [21] Jixuan Ruan, Xiang Fang, Hezi Zhang, Ang Li, Travis Humble, and Yufei Ding. 2025. Powermove: Optimizing compilation for neutral atom quantum computers with zoned architecture. In *Proceedings of the 30th ACM International Conference on Architectural Support for Programming Languages and Operating Systems, Volume 3*. 163–178.
- [22] Ludwig Schmid, Sunghye Park, and Robert Wille. 2024. Hybrid circuit mapping: Leveraging the full spectrum of computational capabilities of neutral atom quantum computers. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [23] Yannick Stade, Lukas Burgholzer, and Robert Wille. 2025. Search Smarter, Not Harder: A Scalable, High-Quality Zoned Neutral Atom Compiler. *arXiv preprint arXiv:2512.13790* (2025).
- [24] Yannick Stade, Wan-Hsuan Lin, Jason Cong, and Robert Wille. 2025. Routing-aware placement for zoned neutral atom-based quantum computing. In *2025 IEEE/ACM International Conference On Computer Aided Design (ICCAD)*. IEEE, 1–9.
- [25] Lin Su, Alexander Douglas, Michal Szurek, Anne H Hébert, Aaron Krahn, Robin Groth, Gregory A Phelps, Ognjen Marković, and Markus Greiner. 2025. Fast single atom imaging for optical lattice arrays. *Nature Communications* 16, 1 (2025), 1017.
- [26] Daniel Bochen Tan, Wan-Hsuan Lin, and Jason Cong. 2025. Compilation for dynamically field-programmable qubit arrays with efficient and provably near-optimal scheduling. In *Proceedings of the 30th Asia and South Pacific Design Automation Conference*. 921–929.
- [27] Hanrui Wang, Pengyu Liu, Daniel Bochen Tan, Yilian Liu, Jiaqi Gu, David Z Pan, Jason Cong, Umut A Acar, and Song Han. 2024. Atomique: A quantum compiler for reconfigurable neutral atom arrays. In *2024 ACM/IEEE 51st Annual International Symposium on Computer Architecture (ISCA)*. IEEE, 293–309.
- [28] Hanrui Wang, Daniel Bochen Tan, Pengyu Liu, Yilian Liu, Jiaqi Gu, Jason Cong, and Song Han. 2024. Q-pilot: Field programmable qubit array compilation with flying ancillas. In *Proceedings of the 61st ACM/IEEE Design Automation Conference*. 1–6.
- [29] Ming Zhong, Zhemin Zhang, Xiangyu Ren, Chenghong Zhu, Siyuan Niu, and Zhiding Liang. 2025. A Compilation Framework for Quantum Circuits with Mid-Circuit Measurement Error Awareness. *arXiv preprint arXiv:2511.10921* (2025).
- [30] Yidong Zhou, Lingyi Kong, Yifeng Peng, and Zhiding Liang. 2026. Reinforcement Learning for Enhanced Advanced QEC Architecture Decoding. In *2026 31st Asia and South Pacific Design Automation Conference (ASP-DAC)*. 83–89. doi:10.1109/ASPDAC66049.2026.11420491